Pontificia Universidad JAVERIANA Bogotá

Ingeniería de Sistemas

PROPUESTA PARA PROYECTO DE GRADO

TÍTULO

Kit Pagos Colombia: ecosistema unificado de herramientas para la integración de pasarelas de pago en el contexto colombiano

OBJETIVO GENERAL

Desarrollar una solución de software para la unificación de la integración de pasarelas de pago en el contexto colombiano, con el fin de simplificar el proceso de desarrollo y evaluar su impacto en términos de complejidad y eficiencia.

ESTUDIANTE(S)

Andres Henao Niño

Documento	Celular	Teléfono fijo	Correo Javeriano
cc. 1033097753	3150552887	0	ahenaon@javeriana.edu.co

Joan Emmanuel Orduz Chia

Documento	Celular	Teléfono fijo	Correo Javeriano
cc. 1054283826	3123164634	8101122	joan_orduz@javeriana.edu.co

Joshua Prieto Zambrano

Documento	Celular	Teléfono fijo	Correo Javeriano
cc. 1013259685	3118468556	0	prietojoshua@javeriana.edu.co

David Estevan Rodriguez Jurado

Documento	Celular	Teléfono fijo	Correo Javeriano
cc. 1141714089	3214334425	0	rodriguezj.david@javeriana.edu.co

DIRECTOR

Ing. Jaime Andrés Pavlich Mariscal

Documento	Celular	Teléfono fijo	Correo Javeriano	Empresa donde trabaja y cargo
cc. 0	0	0 ext 4731	jpavlich@javeriana.edu.co	Pontificia Universidad Javeriana; Profesor Titular - Consejero académico Departamento de Ingeniería Sistemas

3/13/2026

Contenido

1 Visión global **2**

1.1 Antecedentes, problema y solución propuesta 2
1.1.1 Descripción de la problemática u oportunidad *2*
1.1.2 Formulación del problema *2*
1.1.3 Propuesta de solución *2*
1.1.4 Justificación de la solución *2*
1.2 Descripción general del proyecto 2
1.2.1 Objetivo general *2*
1.2.2 Objetivos Específicos *3*
1.3 Entregables, estándares utilizados y justificación 4

2 Análisis de impacto **5**

3 Proceso **6**

3.1 Fase metodológica 1 6
3.1.1 Método *6*
3.1.2 Actividades *6*
3.1.3 Resultados esperados *6*
3.2 Fase metodológica 2 7
3.2.1 Método *7*
3.2.2 Actividades *7*
3.2.3 Resultados esperados *7*
3.3 Fase metodológica 3 7
3.3.1 Método *7*
3.3.2 Actividades *7*
3.3.3 Resultados esperados
3.4 Fase metodológica 4 7
3.3.1 Método *7*
3.3.2 Actividades *7*
3.3.3 Resultados esperados
3.5 Fase metodológica 5 7
3.3.1 Método *7*
3.3.2 Actividades *7*
3.3.3 Resultados esperados *7*

3.6 Fase metodológica 6 7
3.3.1 Método *7*
3.3.2 Actividades *7*
3.3.3 Resultados esperados

4 Aspectos generales del proyecto **8**

4.1 Compromiso de apoyo de la Institución 8
4.2 Derechos patrimoniales 8

5 Marco teórico **9**

5.1 Fundamentos y conceptos relevantes para el proyecto. 9
5.2 Trabajos importantes en el área 9

6 Referencias **10**

1 Visión global

1.1 Antecedentes, problema y solución propuesta

1.1.1 Descripción de la problemática u oportunidad

El comercio electrónico en Colombia ha crecido de forma sostenida durante los últimos años. Según la Cámara Colombiana de Comercio Electrónico, en 2025 las transacciones digitales superaron los 140 billones de pesos («Informe de comercio minorista / Diciembre 2025», s. f.), y la tendencia apunta a que este número seguirá aumentando en los próximos años. Este crecimiento ha llevado a que cada vez más emprendedores, *startups* y equipos de desarrollo enfrenten la necesidad de integrar soluciones de pago en sus aplicaciones.

El mercado colombiano de pasarelas de pago está actualmente dominado por tres actores principales: Wompi, PayU Latam y Mercado Pago. A estas se suma Kushki, plataforma de origen latinoamericano con operación activa en Colombia, que representa una alternativa de integración regional relevante para proyectos con proyección más allá del mercado local. Cada una de estas plataformas ofrece su propia interfaz de programación de aplicaciones (*Application Programming Interface*, API) con formatos de autenticación distintos, estructuras de respuesta diferentes, mecanismos de notificación asíncrona por HTTP (*webhook*) particulares y comportamientos específicos ante errores y reintentos.

Esta fragmentación representa un problema concreto para los desarrolladores: cada integración con una pasarela requiere estudiar documentación propia, implementar lógica de autenticación diferente, normalizar manualmente los formatos de respuesta y manejar los webhooks de forma independiente. Si un proyecto necesita más de una pasarela —lo que es común en aplicaciones que buscan mayor cobertura o disponibilidad— el tiempo invertido se multiplica. A esto se suma la dificultad de hacer pruebas durante el desarrollo: el entorno de pruebas (*sandbox*) de las pasarelas reales tiene límites de uso, requieren credenciales específicas y no siempre permiten simular escenarios de error de forma controlada.

En ese contexto, la oportunidad está en construir una herramienta que centralice y simplifique este proceso. El ecosistema de pagos colombiano carece de un kit de desarrollo que unifique las principales pasarelas bajo una interfaz común, que facilite las pruebas locales sin depender de conexiones externas y que genere datos de prueba válidos para el contexto local. Esa brecha es la que motiva este proyecto.

1.1.2 Formulación del problema

Los desarrolladores colombianos que necesitan integrar múltiples pasarelas de pago en sus aplicaciones deben enfrentarse a un ecosistema fragmentado, en el que cada proveedor define sus propias convenciones de API, autenticación, manejo de errores y notificaciones asíncronas. Esto obliga a implementar desde cero una lógica de integración específica para

cada pasarela, generando duplicación de esfuerzo, mayor probabilidad de errores y tiempos de desarrollo significativamente más largos que se multiplican con cada proveedor adicional.

Trabajos previos como el de Manuaba e Hisyam (2022), que integraron dos pasarelas de pago indonesias (Midtrans y Xendit) en un escenario de pagos divididos, demuestran la viabilidad técnica de unificar múltiples proveedores bajo una sola capa de integración. Sin embargo, ese enfoque se limitó a un prototipo específico para un mercado particular, sin contemplar un framework reutilizable, herramientas de simulación web ni acceso consolidado a datos de prueba adaptados al contexto local. Aunque existen soluciones de unificación en otros mercados —como Hyperswitch a nivel global o el modelo documentado por Manuaba y Hisyam (2022) en Indonesia—, no existe en Colombia un framework que unifique este proceso de manera reutilizable y open source.

Pregunta de investigación: ¿Cómo desarrollar un framework de integración compuesto por un SDK con API normalizada, una API web de simulación desplegada y documentación de datos de prueba colombianos que simplifique la integración de múltiples pasarelas de pago en aplicaciones de software desarrolladas en Colombia?

Propuesta de solución

Se propone el desarrollo del framework Kit Pagos Colombia, un conjunto de herramientas orientadas a facilitar el proceso de integración de pasarelas de pago utilizadas en el contexto colombiano. La propuesta busca explorar la viabilidad de un enfoque que unifique distintos proveedores bajo una capa común de interacción para desarrolladores.

El framework estaría compuesto por tres componentes principales que funcionan de manera complementaria:

Componente 1 – SDK o API unificada

Consistiría en una biblioteca de desarrollo implementada cuyo propósito sería abstraer parte de las diferencias existentes entre las APIs de proveedores como Wompi, PayU, Mercadopago y Kushki. A través de esta capa, los desarrolladores podrían interactuar con varias pasarelas utilizando una interfaz relativamente homogénea para operaciones frecuentes como la creación de pagos, consulta de estados o gestión de notificaciones. Durante el desarrollo del proyecto se evaluará la utilización de patrones de diseño comunes en integraciones externas, así como mecanismos de manejo de errores y reintentos que permitan mejorar la estabilidad de las comunicaciones con los proveedores.

En cuanto a la seguridad, el framework incorporará una capa de seguridad en la comunicación entre el cliente y el servidor, garantizando el manejo seguro de credenciales y datos durante el proceso de integración. Es importante aclarar que la capa de seguridad propia de cada pasarela (autenticación con tokens, cifrado de datos de pago, manejo de PCI-DSS) no será modificada ni intervenida, dado que ya está establecida y gestiona datos sensibles bajo

sus propios estándares certificados. El framework actúa como intermediario que delega correctamente hacia cada proveedor sin interferir en sus mecanismos de seguridad internos.

Componente 2 – Documentación de datos de prueba:

El framework podría consolidar y exponer a través de su documentación los datos de prueba ya provistos por cada pasarela (tarjetas de prueba, números de identificación válidos para sandbox, referencias de pago, etc.). Este componente estará orientado a facilitar el acceso centralizado a la información de prueba disponible en cada proveedor, reduciendo el tiempo que los desarrolladores invierten en localizar estos datos dispersos en múltiples fuentes de documentación.

Componente 3 – API web de simulación desplegada

Se plantea la construcción de una API web desplegada que exponga endpoints que repliquen el comportamiento de las pasarelas de pago para escenarios de prueba. Este componente tendrá como objetivo facilitar las pruebas durante el desarrollo, reduciendo la dependencia de entornos sandbox externos que pueden presentar limitaciones de acceso, restricciones de uso o dificultades para reproducir ciertos escenarios de error de forma controlada. Al estar desplegada, la API será accesible sin requerir configuración local adicional.

En conjunto, estos componentes buscan conformar una base experimental para analizar hasta qué punto una aproximación unificada mediante framework puede simplificar el trabajo de integración de pasarelas de pago en proyectos de software colombianos.

1.1.3 Justificación de la solución

La propuesta surge de la observación de que el proceso de integración de pasarelas de pago suele implicar una cantidad considerable de trabajo repetitivo para los equipos de desarrollo. Cada proveedor define sus propios formatos de autenticación, estructuras de respuesta y mecanismos de notificación, lo que obliga a implementar lógica específica para cada integración.

Explorar la construcción de una capa de abstracción común podría contribuir a reducir parte de esa complejidad, permitiendo que los desarrolladores trabajen con un conjunto más consistente de operaciones.

Por otro lado, el proceso de pruebas en integraciones de pago suele depender de entornos sandbox proporcionados por los mismos proveedores. Estos entornos pueden presentar limitaciones de acceso, restricciones de uso o dificultades para reproducir ciertos escenarios. En este contexto, una API web de simulación desplegada podría facilitar el desarrollo al permitir pruebas más controladas durante etapas tempranas del proyecto.

Adicionalmente, consolidar el acceso a los datos de prueba ya disponibles en cada pasarela en lugar de dispersarlos en múltiples fuentes puede representar un apoyo significativo para los desarrolladores, reduciendo el tiempo de búsqueda y configuración inicial.

En conjunto, la propuesta busca analizar si la combinación de estas herramientas puede contribuir a mejorar la experiencia de desarrollo en proyectos que requieren integrar múltiples pasarelas de pago.

1.2 Descripción general del proyecto

Esta sección presenta los aspectos centrales que definen el alcance y propósito del proyecto. El framework Kit Pagos Colombia busca explorar un enfoque de herramientas que facilite el desarrollo de integraciones de pago en el contexto colombiano combinando rigor técnico con una experiencia de uso accesible para desarrolladores con distintos niveles de experiencia.

1.2.1 Objetivo general

Desarrollar el Kit Pagos Colombia, para simplificar la integración de múltiples pasarelas de pago en el contexto colombiano.

1.2.2 Objetivos Específicos

- **Análisis**

  Analizar las características técnicas, mecanismos de integración y consideraciones de seguridad de las principales pasarelas de pago utilizadas en el contexto colombiano.

- **Diseño**

  Diseñar la arquitectura de un framework de integración que permita unificar la interacción con múltiples pasarelas de pago, aplicando patrones de diseño y principios de extensibilidad.

- **Desarrollo**

  Desarrollar un prototipo funcional del framework que implemente una interfaz unificada para la integración con múltiples pasarelas de pago, incluyendo mecanismos de comunicación segura y los elementos necesarios para su integración por parte de desarrolladores.

- **Simulacióna**

  Implementar una API que permita simular escenarios de interacción con pasarelas de pago, facilitando procesos de prueba durante el desarrollo.

- **Datos de prueba**

  Consolidar y documentar los mecanismos de acceso a los datos de prueba provistos por cada pasarela de pago en el contexto colombiano, facilitando su uso centralizado.

- **Evaluación**

  Evaluar el framework mediante su aplicación en proyectos prototípicos definidos, especificando sus características y midiendo métricas asociadas a su implementación y nivel de abstracción, con el fin de analizar su impacto frente a enfoques tradicionales.

1.3 Entregables, estándares utilizados y justificación

El proyecto producirá cinco artefactos principales. Cada uno responde a uno o varios objetivos específicos definidos en la sección 1.2.2 y se apoya en estándares de ingeniería consolidados que fundamentan las decisiones de diseño, construcción y evaluación adoptadas a lo largo del proyecto.

Entregable	**Estándares asociados**	**Justificación**
Prototipo funcional del SDK con interfaz unificada para Wompi, PayU, Mercadopago y Kushki	Cockburn, A. (2024). *Hexagonal Architecture Explained*; Gamma, E., Helm, R., Johnson, R., & Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*; Hohpe, G. & Woolf, B. (2003). *Enterprise*	La arquitectura hexagonal establece el estilo estructural del SDK, garantizando que el núcleo de lógica de pagos sea independiente de los proveedores. Los patrones GoF (Strategy, Facade y Factory) fundamentan las decisiones de diseño de la interfaz unificada y los adaptadores. Los patrones de integración empresarial de Hohpe y Woolf justifican el mecanismo de normalización de respuestas y el manejo de comunicaciones asíncronas entre el framework y las pasarelas.

	Integration Patterns	
API de simulación desplegada con endpoints que replican el comportamiento de las pasarelas	Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*; Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing*	Los principios REST de Fielding fundamentan el diseño de la interfaz de la API de simulación, garantizando una interfaz coherente, sin estado y desacoplada de cualquier cliente. Los criterios de Ammann y Offutt justifican la cobertura de escenarios que debe exponer la API (éxito, rechazo, error de red y notificaciones asíncronas) y el entorno controlado equivalente a un sandbox.
Documentación centralizada de datos de prueba por pasarela	Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing*	La noción de entorno sandbox como espacio de verificación aislado, definida por Ammann y Offutt, justifica la necesidad de consolidar los datos de prueba de cada proveedor en un formato accesible y estructurado, reduciendo la dependencia de fuentes dispersas durante el desarrollo.
Informe comparativo de métricas de evaluación (WMC, CBO, RFC)	Chidamber, S. R. & Kemerer, C. F. (1994). *A metrics suite for object oriented design*. IEEE Transactions on Software Engineering, 20(6), 476–493	Las métricas CK (Chidamber y Kemerer) proveen una herramienta cuantitativa y reproducible para comparar la complejidad, el acoplamiento y la superficie de interacción del código de integración con y sin el framework. Su uso garantiza que la evaluación del artefacto no dependa exclusivamente de observaciones subjetivas.
Documento final del trabajo de grado y repositorio público con documentación de uso	Peffers, K., Tuunanen, T., Rothenberger, M., & Chatterjee, S. (2007). *A design science research methodology for information systems research*. Journal of	DSR exige que los artefactos producidos y las lecciones aprendidas sean comunicados de forma rigurosa a las audiencias relevantes. El documento de grado y el repositorio público materializan esta exigencia, presentando el proceso de construcción, los resultados de la evaluación y las condiciones bajo las cuales el framework resulta útil.

	Management Information Systems, 24, 45–77	

2 Análisis de impacto

A corto plazo, el beneficio directo recae sobre desarrolladores individuales y equipos pequeños que enfrentan por primera vez la integración de una pasarela de pago colombiana en sus proyectos. Para este perfil de usuario, la existencia de un SDK con interfaz unificada, una API de simulación accesible sin configuración adicional y documentación centralizada de datos de prueba reduciría la fricción inicial del proceso de integración. Esta reducción no elimina la complejidad inherente al ecosistema de pagos, pero sí concentra en un único punto de entrada información y lógica que actualmente se encuentra dispersa entre cuatro fuentes de documentación distintas. El impacto en este aspecto es operativo: menos tiempo invertido en lectura de documentación, menos errores derivados de la configuración manual de credenciales de sandbox y mayor control sobre los escenarios de prueba durante las etapas tempranas del desarrollo.

A mediano plazo, si el framework logra adoptarse más allá del contexto evaluativo del proyecto, el beneficio potencial se extiende hacia el ecosistema de desarrollo de software colombiano en sentido más amplio. Colombia cuenta con una comunidad activa de desarrolladores web y móviles que atiende la demanda de comercio electrónico local, demanda que, según los datos de la Cámara Colombiana de Comercio Electrónico citados en la sección 1.1.1, supera ya los 140 billones de pesos en transacciones anuales. Para esa comunidad, disponer de una herramienta open source mantenida y documentada podría representar una reducción real en el tiempo de incorporación de funcionalidades de pago en nuevos proyectos. Adicionalmente, el repositorio público del framework y su documentación pueden constituir un recurso de referencia para estudiantes y programas académicos que aborden la enseñanza de integración de sistemas, dado que el caso de las pasarelas de pago colombianas es un escenario concreto, técnicamente representativo y de alta relevancia práctica para el mercado local.

A largo plazo, el impacto del proyecto depende de condiciones que la propuesta no puede garantizar: la continuidad del mantenimiento del framework una vez concluido el trabajo de grado, la evolución de las APIs de los proveedores integrados y la disposición de la comunidad de desarrollo a adoptar y contribuir a una herramienta de este tipo. Si estas condiciones se cumplen, el Kit Pagos Colombia podría evolucionar de prototipo académico a referencia técnica activa dentro del ecosistema fintech colombiano, facilitando que proyectos con menor capacidad de inversión en desarrollo puedan incorporar múltiples opciones de pago sin duplicar esfuerzo. En este escenario, el impacto social indirecto se manifestaría en una mayor facilidad para que emprendimientos y pequeñas empresas colombianas accedan a infraestructura de cobro digital diversificada, lo que contribuiría marginalmente a la reducción de barreras de entrada al comercio electrónico formal.

Es importante señalar que estos impactos son condicionales y proporcionales al alcance real del proyecto. El framework no modifica las condiciones estructurales del ecosistema de pagos

colombiano, no interviene en los costos de procesamiento de las pasarelas ni resuelve problemas de acceso financiero más amplios. Su contribución es técnica y está delimitada al problema específico de la fragmentación en el proceso de integración, tal como fue definido en la sección 1.1.2.

3 Proceso

El desarrollo del proyecto se lleva a cabo mediante Design Science Research (DSR), metodología seleccionada como marco principal dado que está diseñada para proyectos cuyo resultado central es la construcción y evaluación de un artefacto que resuelve un problema real. Esta metodología estructura el proyecto en fases que van desde la identificación del problema hasta la comunicación de resultados, exigiendo en cada etapa tanto rigor académico como validación práctica del artefacto construido. DSR resulta especialmente adecuada para este proyecto dado que el framework no es un medio para responder una pregunta teórica, sino el aporte central de la investigación, y su evaluación en contextos reales de integración de pasarelas de pago constituye parte fundamental del proceso investigativo.

3.1 Fase metodológica 1 - Identificación y definición del problema

Esta primera etapa surge de la necesidad de comprender el entorno técnico en el cual se desarrollará el proyecto. No se limita a un análisis técnico superficial, sino que busca profundizar en las dificultades que enfrentan los desarrolladores en Colombia al integrar sistemas de pago. La fase consiste en analizar el ecosistema actual y a sus actores principales (Wompi, PayU, Mercado Pago y Kushki), con el fin de identificar puntos comunes y las barreras técnicas que imponen sus diferencias. El objetivo final es transformar dicha fragmentación en una hoja de ruta clara para el diseño de la solución.

3.1.1 Método

Para estructurar esta exploración, se adaptó la Ingeniería de Requisitos (basada en el estándar IEEE 29148). En este enfoque, las fuentes de información no son personas, sino las plataformas mismas y su documentación técnica oficial. La metodología se ajustó para tratar cada API como un conjunto de necesidades técnicas por resolver. De esta manera, el proceso trasciende la lectura documental para convertirse en una labor de normalización: se toma la complejidad técnica de cada pasarela y se convierte en los requisitos funcionales que darán vida a la arquitectura del Kit Pagos Colombia.

3.1.2 Actividades

Para garantizar la efectividad del proceso, se llevarán a cabo las siguientes actividades:

- **Inmersión en la documentación:** Realizar un estudio profundo de las especificaciones de Wompi, PayU, Mercado Pago y Kushki para comprender sus protocolos de comunicación.
- **Mapeo de diferencias:** Identificar las discrepancias en los formatos de datos, especialmente en la gestión de estados de transacciones (éxito, fallo o pendiente).

- **Análisis de seguridad y notificaciones:** Evaluar los flujos de información y el funcionamiento de los avisos automáticos (webhooks) para garantizar la confiabilidad del framework.
- **Definición de la base del framework:** Consolidar los hallazgos en una especificación técnica de funciones y características que la herramienta debe estandarizar.

3.1.3 Resultados esperados

- **Mapa del ecosistema:** Un documento detallado que funciona como guía técnica, exponiendo las similitudes y retos de cada pasarela analizada. Este resultado constituye el insumo clave para asegurar el **cumplimiento total del primer objetivo específico** de la investigación.
- **Repositorio de parámetros de prueba:** Una recopilación organizada de datos, tarjetas de prueba y tokens de acceso. Aunque se considera un resultado intermedio, es fundamental para validar el progreso de la investigación y asegurar una base sólida antes de iniciar la escritura del código de la solución.

3.2 Fase metodológica 2 - Definición de Objetivos

Esta fase tiene como propósito delimitar el alcance del framework y establecer los criterios que guiarán su diseño y construcción. A partir del análisis del ecosistema realizado en la fase anterior, se definen los componentes que conformarán la solución, las pasarelas que serán integradas y los criterios con los que se evaluará el éxito del proyecto.

3.2.1 Método

Esta fase se apoya en la etapa de definición del problema y objetivos de la metodología principal utilizada, Design Science Research (DSR), en la cual se propone traducir la problemática identificada y lo propuesto en requisitos concretos con el fin de construir el artefacto planteado. Para estructurar este proceso de especificación, se toma como referencia el estándar IEEE 830-1998 (IEEE Recommended Practice for Software Requirements Specifications), que establece las convenciones para documentar los requisitos funcionales y no funcionales de un sistema de software de forma organizada, verificable y comprensible para todas las partes involucradas. Bajo este marco, se definen los componentes funcionales del framework, las pasarelas objetivo y los criterios de evaluación que permitan verificar si el artefacto resuelve efectivamente el problema planteado.

3.2.2 Actividades

Para que este proceso sea efectivo, se seguirán los siguientes pasos:

- **Definición de componentes:** Se establecerán las partes que conformarán el framework y el rol que cumple cada una dentro de la solución, asegurando que trabajen de forma complementaria.
- **Selección de pasarelas objetivo:** Se identificarán y justificarán las pasarelas colombianas que serán integradas, basándose en los hallazgos del análisis previo.
- **Establecimiento de requisitos funcionales:** Se traducirá lo aprendido en la fase anterior en una lista concreta de operaciones que el framework debe soportar por cada pasarela, como creación de pagos, consulta de estados y manejo de webhooks.
- **Definición de criterios de evaluación:** Se determinarán las métricas con las que se medirá si el framework resuelve efectivamente el problema, incluyendo indicadores como tiempo de integración y cobertura funcional.
- **Caracterización de proyectos prototípicos:** Se especificarán las características que deben tener los proyectos sobre los que se evaluará el framework durante la fase de demostración.

3.2.3 Resultados esperados

- **El plano del framework (Documento de alcance):** Documento que consolida los componentes definidos, las pasarelas seleccionadas y los requisitos funcionales por proveedor. Constituye el insumo directo para iniciar el diseño y desarrollo del artefacto.
- **La vara de medición (Criterios de evaluación):** Definición clara de los indicadores con los que se verificará si el framework cumple su propósito. Aunque es un resultado interno, resulta fundamental establecerlo en esta fase para que la evaluación final no sea subjetiva.
- **Backlog inicial (Resultado interno):** Lista priorizada de funcionalidades que guiará el desarrollo, constituyendo el punto de partida para la construcción ordenada del framework.

3.3 Fase metodológica 3 – Diseño

Esta fase transforma los insumos generados en las fases anteriores en decisiones arquitectónicas concretas y verificables. Su propósito no es producir código, sino establecer los contratos, estructuras y límites dentro de los cuales el desarrollo posterior podrá avanzar con coherencia. El hilo conductor de esta fase es la arquitectura hexagonal, que impone una disciplina estructural orientada a la extensibilidad del framework y que determina cómo se relacionan sus componentes entre sí y con el exterior.

3.3.1 Método

El método de esta fase se fundamenta en la arquitectura hexagonal, también conocida como arquitectura de puertos y adaptadores, como estilo arquitectónico central. Este estilo organiza el sistema alrededor de un núcleo de lógica de negocio independiente que se comunica con el exterior únicamente a través de puertos bien definidos. Cada pasarela de pago, Wompi, PayU, Mercadopago y Kushki, se concibe como un adaptador que traducirá el contrato del puerto a las convenciones específicas de ese proveedor, sin que el núcleo tenga conocimiento de los detalles de implementación de ninguno de ellos.

Esta elección arquitectónica responde directamente a la naturaleza del problema: el ecosistema colombiano de pasarelas de pago es fragmentado y está en constante evolución, por lo que la solución debe anticipar la necesidad de incorporar nuevos proveedores o modificar adaptadores existentes sin intervenir en la lógica central del framework. La separación entre el núcleo y sus adaptadores garantiza que la adición de una nueva pasarela implique únicamente la implementación de un nuevo adaptador que cumpla el contrato del puerto, sin afectar los componentes ya desarrollados ni los proyectos que consumen el framework.

Adicionalmente, se adoptarán patrones de diseño del catálogo GoF donde su aplicación resulte pertinente. En particular, el patrón Strategy permitirá encapsular el comportamiento específico de cada pasarela como una estrategia intercambiable, el patrón Facade proveerá la interfaz unificada que los desarrolladores consumirán, y el patrón Factory facilitará la instanciación de los adaptadores según el proveedor seleccionado. Para el componente de la API de simulación, el diseño seguirá los principios REST con el fin de garantizar una interfaz coherente, predecible y desacoplada de cualquier implementación cliente.

3.3.2 Actividades

- **Definición del modelo de dominio unificado:** Identificar y modelar las entidades centrales del dominio de pagos (transacción, estado de pago, notificación asíncrona, error normalizado) que constituirán el núcleo de la arquitectura hexagonal, independiente de cualquier pasarela específica.
- **Diseño de los puertos del framework:** Definir los contratos de los puertos de entrada y salida del núcleo hexagonal, especificando las interfaces que cada adaptador deberá implementar para conectar una pasarela al framework. Esto incluye los puertos para creación de pagos, consulta de estado y recepción de webhooks.
- **Especificación de los adaptadores por pasarela:** Determinar, para cada pasarela soportada (Wompi, PayU, Mercadopago y Kushki), la correspondencia entre el contrato del puerto y las convenciones de autenticación, estructura de solicitud y formato de respuesta propias de ese proveedor. Esta actividad no implica aún su implementación, sino la definición del contrato que cada adaptador deberá cumplir.

- **Diseño de la API de simulación:** Definir los endpoints que la API REST de simulación deberá exponer, los escenarios que cada uno replicará (éxito, rechazo, error de red y notificaciones asíncronas) y la estructura de sus respuestas, garantizando coherencia con los contratos de puerto ya definidos.
- **Definición de la arquitectura de seguridad en la comunicación:** Especificar los mecanismos que garantizarán el manejo seguro de credenciales y la comunicación cifrada entre el cliente y el framework, delimitando con precisión la responsabilidad del framework frente a los mecanismos de seguridad internos de cada pasarela.

3.3.3 Resultados esperados

- **Modelo de dominio unificado documentado**, que describa las entidades centrales del núcleo y sus relaciones, con independencia de cualquier pasarela específica. Este artefacto constituye la base conceptual sobre la que se construirán los puertos y los adaptadores.
- **Contratos de los puertos del framework**, que especifiquen con precisión las interfaces que cada adaptador deberá cumplir para conectarse al núcleo. Este artefacto es el insumo directo para la fase de desarrollo y el criterio de verificación de la corrección de cada adaptador.
- **Especificación de los adaptadores por pasarela**, que describa la correspondencia entre los contratos del puerto y las convenciones técnicas de Wompi, PayU, Mercadopago y Kushki, sin que esta especificación constituya aún código ejecutable.
- **Diseño documentado de la API de simulación**, que incluya la definición de endpoints, parámetros de entrada, estructuras de respuesta y escenarios cubiertos, listo para orientar su implementación en la fase siguiente.
- **Arquitectura hexagonal documentada del framework**, que integre el diagrama de componentes del núcleo, los contratos de los puertos y la descripción de los adaptadores planificados. Este artefacto evidencia que la decisión arquitectónica fue llevada a un diseño concreto, revisable y extensible.

3.4 Fase metodológica 4 – Desarrollo

Esta fase materializa las decisiones de diseño tomadas en la fase anterior mediante la construcción iterativa e incremental de los tres componentes del framework. Su punto de partida son los contratos de puerto, el modelo de dominio y las especificaciones de los adaptadores producidos en la fase de diseño; su resultado es un artefacto funcional, verificable y distribuible. La fase no opera en un único sentido: la implementación puede revelar restricciones o posibilidades que retroalimenten y refinen decisiones de diseño previas, de modo que ambas fases mantienen un vínculo de ajuste mutuo hasta que los artefactos convergen en una versión estable.

3.4.1 Método

El desarrollo se realiza de forma iterativa e incremental, siguiendo el orden lógico de dependencias que impone la arquitectura hexagonal. La primera iteración construye el núcleo del framework y sus contratos de puerto, que actúan como la base sobre la cual todos los demás componentes dependen. La segunda iteración implementa los adaptadores de cada pasarela de forma independiente entre sí, verificando en cada caso que el adaptador cumple el contrato del puerto sin modificar el núcleo. La tercera iteración desarrolla la API de simulación y consolida la documentación de datos de prueba, que constituirán el entorno controlado para la verificación interna. Cada iteración produce artefactos ejecutables que pueden verificarse de forma aislada antes de avanzar a la siguiente.

3.4.2 Actividades

- **Implementación del núcleo del framework:** Desarrollar la lógica central del framework (la capa de normalización de respuestas, el mecanismo de selección de adaptador, el manejo unificado de errores y los flujos de reintento), manteniendo esta lógica completamente desacoplada de los detalles de implementación de cada pasarela.
- **Implementación de los adaptadores por pasarela:** Desarrollar un adaptador independiente para cada pasarela soportada (Wompi, PayU, Mercadopago y Kushki), encargado de traducir el contrato del puerto a las convenciones de autenticación, estructura de solicitud y formato de respuesta propias de ese proveedor.
- **Desarrollo de la API de simulación:** Implementar la API REST cuyo diseño fue definido en la fase anterior, desplegándola de forma accesible y verificando que sus endpoints replican los comportamientos de las pasarelas en los escenarios de prueba identificados.
- **Consolidación de la documentación de datos de prueba:** Centralizar y estructurar los datos de prueba provistos por cada pasarela (tarjetas de prueba, documentos de identidad válidos para sandbox, referencias de pago y credenciales de entorno) en un formato accesible y unificado como parte de la documentación del framework.
- **Implementación de la capa de seguridad en la comunicación:** Incorporar los mecanismos que garanticen el manejo seguro de credenciales y la comunicación cifrada entre el cliente y el servidor, delegando correctamente hacia los mecanismos de seguridad internos de cada pasarela sin interferir en ellos.
- **Desarrollo de pruebas internas del framework:** Implementar pruebas unitarias sobre el núcleo y pruebas de integración sobre los adaptadores, utilizando la API de simulación como entorno controlado, con el fin de verificar el comportamiento del framework antes de su evaluación formal.
- **Empaquetado y configuración del proyecto como librería open source:** Preparar el framework para su distribución como librería reutilizable, incluyendo la

configuración del repositorio, el archivo de manifiesto, la estructura de dependencias y la documentación de instalación orientada al desarrollador.

3.4.3 Resultados esperados

- **Prototipo funcional del framework** con una interfaz unificada para las operaciones de creación de pagos, consulta de estado y gestión de notificaciones, con adaptadores operativos para Wompi, PayU, Mercadopago y Kushki. Este resultado corresponde al cumplimiento del objetivo específico de Desarrollo.
- **API de simulación desplegada y accesible**, con endpoints documentados que repliquen los comportamientos principales de las pasarelas de pago sin requerir credenciales externas. Este resultado corresponde al cumplimiento del objetivo específico de Simulación.
- **Documentación centralizada de datos de prueba**, estructurada por pasarela y tipo de dato, accesible desde el repositorio del proyecto. Este resultado corresponde al cumplimiento del objetivo específico de Datos de prueba.
- **Suite de pruebas internas ejecutables** sobre el núcleo y los adaptadores del framework, que validen el comportamiento esperado en escenarios controlados y sirvan como base para la evaluación formal de la fase siguiente.

3.3 Fase metodológica 5 - Demostración y evaluación

Esta fase consiste en la demostración y la evaluación del framework. Dentro de la naturaleza del proceso DSR lo más óptimo para el proyecto es aplicar el kit en proyectos prototípicos y hacer la medición, ya que la observación de las métricas ocurre a la par de la misma integración.

3.3.1 Método

La evaluación del framework se estructura a partir de los procesos de verificación y validación definidos en el estándar IEEE 1012 (*Standard for System, Software, and Hardware Verification and Validation*), adaptados al contexto de un artefacto de software académico. IEEE 1012 distingue entre verificación (determinar si el artefacto fue construido correctamente según sus especificaciones) y validación (determinar si el artefacto resuelve efectivamente el problema para el que fue diseñado). En esta fase, ambas dimensiones operan simultáneamente: la verificación se apoya en las métricas CK calculadas sobre el código, y la validación ocurre al aplicar el framework en proyectos prototípicos reales y contrastar los resultados con el enfoque tradicional.

Para operacionalizar esta evaluación, se adopta adicionalmente la fase de *Demonstration* de DSR (Peffers et al., 2007), que establece que el artefacto debe ser aplicado en un contexto representativo del problema original. Cada proyecto prototípico actúa simultáneamente como

escenario de uso y como caso de medición, de modo que la demostración y la evaluación no se ejecutan en momentos separados sino que se retroalimentan durante la misma integración.

La medición cuantitativa se realiza mediante tres métricas del conjunto propuesto por Chidamber y Kemerer (1994). Para cada prototipo se ejecutará primero una integración directa con las APIs nativas de los proveedores como grupo de control, y luego una integración equivalente usando Kit Pagos Colombia, calculando sobre ambas las métricas WMC, CBO y RFC sobre las mismas clases. Las métricas propuestas se definen de la siguiente manera:

WMC (Weighted Methods per Class): Mide la suma de la complejidad ciclomática de los métodos definidos en una clase. En el contexto de este proyecto, un WMC menor en las clases que implementan la integración usando el SDK de Kit Pagos Colombia, comparado con las clases equivalentes en la integración directa, indicaría que la capa de abstracción del framework reduce la complejidad lógica que el desarrollador debe escribir e implementar.

CBO (Coupling Between Object Classes): cuenta cuántas clases externas distintas utiliza una clase determinada. Esta métrica es especialmente relevante para este proyecto: una integración directa con múltiples pasarelas genera un alto acoplamiento porque el código cliente debe conocer y referenciar las clases propias de cada proveedor. Se espera que el uso del SDK reduzca el CBO del código cliente, concentrando el acoplamiento externo en la interfaz unificada del framework en lugar de distribuirlo entre los SDKs individuales de cada pasarela.

RFC (Response For a Class): mide el número de métodos que pueden ser invocados en respuesta a un mensaje recibido por un objeto, incluyendo sus métodos propios y los de las clases que llama. Un RFC menor en el código que utiliza el SDK de Kit Pagos Colombia sugeriría que el desarrollador necesita comprender y manejar una superficie de interacción más reducida para completar una operación de pago, lo que se traduce en menor carga cognitiva durante la integración.

Estas tres métricas serán calculadas sobre las mismas clases en ambos escenarios (es decir integración directa de las pasarelas y a través del framework propuesto) y los valores obtenidos constituirán evidencia cuantitativa del nivel de abstracción y simplificación que ofrece el kit. Adicionalmente, se recogerán observaciones cualitativas sobre la experiencia de uso durante el proceso de integración.

3.3.2 Actividades

- **Definición del conjunto de medición:** Definir el conjunto de clases y componentes sobre los cuales se calcularán las métricas CK (Chidamber y Kemerer) en ambos escenarios de integración.

- **Selección de proyectos prototípicos:** Seleccionar los proyectos prototípicos, garantizando que cubran escenarios de integración representativos del contexto colombiano.
- **Medición del grupo de control:** Ejecutar la integración directa con APIs nativas, calcular WMC, CBO y RFC sobre el código resultante, y registrar los valores obtenidos.
- **Medición con el framework:** Ejecutar la integración equivalente utilizando Kit Pagos Colombia, calcular las mismas métricas sobre el código cliente y registrar los resultados.
- **Validación de componentes de prueba:** Aplicar el servidor de simulación y el componente de datos de prueba durante el desarrollo de los prototipos y documentar su comportamiento.
- **Análisis comparativo:** Comparar los valores de las métricas entre ambos enfoques y analizar las diferencias observadas en términos de complejidad, acoplamiento y superficie de interacción.
- **Registro de limitaciones:** Identificar limitaciones, casos no cubiertos y oportunidades de mejora que retroalimenten el desarrollo del framework en caso de que el proceso iterativo lo requiera.

3.3.3 Resultados esperados

- **Prototipos funcionales integrados:** Dos proyectos prototípicos funcionales que integren pasarelas de pago usando Kit Pagos Colombia.
- **Registro comparativo de métricas:** Tabla de valores WMC, CBO y RFC entre integración directa e integración con el framework, para las clases correspondientes en ambos escenarios.
- **Análisis de impacto del framework:** Análisis de los resultados que permita determinar en qué medida el kit reduce la complejidad y el acoplamiento del código de integración frente al enfoque tradicional.
- **Catálogo de limitaciones:** Listado documentado de limitaciones identificadas durante la demostración.

3.4 Fase metodológica 6 - Comunicación

Esta fase corresponde al cierre del proceso DSR. Su propósito es consolidar y comunicar los hallazgos del proyecto de manera que sean útiles tanto para la comunidad académica como para desarrolladores que puedan beneficiarse del framework. No se trata únicamente de documentar lo que se construyó, sino de presentar de forma honesta qué funcionó, qué no, y bajo qué condiciones el kit resulta útil.

3.4.1 Método

La producción de la documentación del framework se guía por el estándar IEEE 1063 (*Standard for Software User Documentation*), que establece los requisitos mínimos de contenido, estructura y verificación que debe cumplir la documentación técnica de un sistema de software para ser útil a su audiencia objetivo. IEEE 1063 define que la documentación debe cubrir al menos: la descripción del propósito del software, las instrucciones de instalación y configuración, la descripción de las funciones disponibles y su modo de uso, y la referencia a los errores conocidos y sus soluciones. En el contexto de este proyecto, estos requisitos se traducen en la documentación del repositorio público del framework, que debe permitir a un desarrollador externo integrar el kit sin necesidad de contactar a los autores.

Complementariamente, se adopta la fase de *Communication* del ciclo DSR (Peffers et al., 2007), que exige que los artefactos producidos y las lecciones aprendidas sean comunicados de forma rigurosa tanto a la comunidad académica como a los practicantes que puedan beneficiarse del resultado. En este proyecto esa doble audiencia determina dos canales distintos: el documento de trabajo de grado, orientado a la comunidad académica, y el repositorio público, orientado a desarrolladores. La redacción del documento final se realiza de forma paralela a las fases anteriores, incorporando resultados a medida que se producen, para que el documento refleje el proceso real y no una reconstrucción posterior.

3.4.2 Actividades

- **Redacción del documento de grado:** Redactar el documento final del trabajo de grado integrando los resultados de todas las fases metodológicas.
- **Publicación del repositorio:** Publicar el repositorio del framework en un espacio de acceso público, incluyendo documentación de uso, instrucciones de instalación y ejemplos de integración para cada pasarela cubierta.
- **Consolidación de datos de prueba:** Consolidar la documentación de datos de prueba como parte del repositorio, asegurando que sea accesible y comprensible para desarrolladores externos al proyecto.

3.4.3 Resultados esperados

- **Documento de grado aprobado:** Documento de trabajo de grado completo y aprobado.
- **Repositorio público autosuficiente:** Repositorio público del framework con documentación suficiente para que un desarrollador externo pueda integrarlo sin necesidad de contactar a los autores.
- **Documentación de datos de prueba publicada:** Documentación de datos de prueba consolidada y publicada como parte del repositorio.

4 Aspectos generales del proyecto

4.1 Derechos patrimoniales

Los derechos morales del presente trabajo de grado corresponden a los autores del proyecto: Andrés Henao Niño, David Estevan Rodríguez Jurado, Joshua Prieto Zambrano y Joan Emmanuel Orduz Chia, en su calidad de estudiantes de la Pontificia Universidad Javeriana.

Dado que el proyecto se desarrolla de forma independiente, sin vinculación a grupos de investigación, entidades externas ni instituciones que hayan financiado o patrocinado su realización, los derechos patrimoniales sobre el artefacto producido corresponden a sus autores.

El framework Kit Pagos Colombia será distribuido como software de código abierto bajo la licencia Apache License 2.0, la cual permite a cualquier persona usar, modificar y distribuir el software, incluyendo su uso comercial, siempre que se conserve el aviso de derechos de autor y se incluya una copia de la licencia en cualquier distribución derivada. Esta licencia incluye además una cláusula explícita de no agresión en materia de patentes, protegiendo tanto a los autores como a los usuarios del framework frente a reclamaciones de este tipo. La elección de esta licencia responde al carácter open source del proyecto y a su propósito de contribuir al ecosistema de desarrollo de software en Colombia de forma abierta y sostenible.

La Pontificia Universidad Javeriana, en su condición de institución académica en la que se desarrolló el proyecto, conservará el derecho de uso del trabajo de grado con fines académicos y de divulgación científica.

5 Marco teórico

El desarrollo del Kit Pagos Colombia se apoya en un conjunto de conceptos, principios y trabajos previos que proveen la base conceptual necesaria para comprender tanto el problema que se busca resolver como las decisiones técnicas que orientan la solución propuesta. Este capítulo tiene como propósito situar al lector en el contexto teórico del proyecto, de modo que los elementos del framework, su arquitectura, sus componentes y su propósito, puedan entenderse con precisión y sin ambigüedad.

La sección 5.1 presenta los fundamentos y conceptos relevantes para el proyecto, es decir, los términos y principios técnicos que aparecen de forma recurrente a lo largo del documento y cuya comprensión es indispensable para evaluar la pertinencia y coherencia de la solución propuesta. Entre estos se encuentran nociones como pasarela de pago, arquitectura hexagonal, normalización de APIs, patrones de diseño y entornos de prueba, conceptos que no solo definen el vocabulario del proyecto sino que también justifican las decisiones de diseño tomadas en la fase metodológica de construcción.

La sección 5.2 presenta los trabajos importantes en el área, donde se examinan tres aproximaciones desarrolladas en otros contextos que abordan problemáticas similares a las que motivaron este proyecto. El análisis de estas alternativas permite identificar qué ha sido explorado previamente, qué limitaciones presentan esas soluciones frente al contexto colombiano, y de qué manera el Kit Pagos Colombia se diferencia y complementa el estado del arte existente.

5.1 Fundamentos y conceptos relevantes para el proyecto.

A continuación se encuentran conceptos relevantes explicados con el fin de entender la propuesta del Kit Pagos Colombianos.

- Pasarela de pago: Una pasarela de pago es un sistema de software que actúa como intermediario entre un comercio y las entidades financieras, autorizando y verificando transacciones de pago en línea mediante la transmisión segura de datos entre el comprador, el comercio y el banco procesador. (Hisyam & Manuaba, 2022)
- API - Interfaz de programación de aplicaciones: Una API es un contrato de comunicación entre sistemas de software que define cómo un programa puede solicitar servicios o datos a otro, sin necesidad de conocer su funcionamiento interno. En el contexto de pagos digitales, las APIs permiten que una aplicación comercial interactúe con una pasarela de pago para crear transacciones, consultar estados y recibir resultados de forma estandarizada.(Fielding, 2000)
- Framework: En ingeniería de software, un framework se entiende como una estructura compuesta por componentes reutilizables que permiten estandarizar y

agilizar el desarrollo de aplicaciones, apoyándose en principios de reutilización y en el uso de estándares definidos (Amazon Web Services, s. f.).

- Patrones de diseño: Los patrones de diseño son soluciones recurrentes y documentadas a problemas comunes en el diseño de software. No son código reutilizable directamente, sino descripciones o plantillas que guían cómo estructurar componentes para resolver un problema específico en un contexto determinado. (*Patrones de Diseño GoF Por Pagina | PDF*, s. f.).
- GoF: En ingeniería de software, GoF es un acrónimo de "Gang of Four"(Pandilla de cuatro) que hace referencia a los cuatro autores (Erich Gamma, Richard Helm, Ralph Johnson y John Vlisside) del libro de 1994, *Design Patterns: Elements of Reusable Object-Oriented Software.* Definen 23 patrones de diseño óptimos para resolver problemas reiterados en la programación orientada a objetos. (*Patrones de Diseño GoF Por Pagina | PDF*, s. f.).
- Arquitectura hexagonal: La arquitectura hexagonal, propuesta por Alistair Cockburn, es un estilo que organiza una aplicación alrededor de un núcleo de lógica de negocio aislado. Este núcleo se comunica mediante puertos y adaptadores que lo conectan con tecnologías externas como bases de datos, APIs o interfaces de usuario. Su objetivo es mantener el núcleo independiente de cualquier tecnología o proveedor externo. (*Hexagonal architecture pattern - AWS Prescriptive Guidance*, s. f.)
- Normalización de APIs: La normalización de APIs es el proceso de unificar las respuestas, estructuras de datos y comportamientos de múltiples servicios externos bajo un formato común y consistente. En sistemas que integran proveedores heterogéneos, la normalización reduce el acoplamiento entre el consumidor y cada proveedor individual, facilitando la intercambiabilidad de servicios. (Taherkordi et al., 2011)
- Entornos de prueba: Un entorno sandbox en el contexto de APIs de pago es un ambiente de simulación aislado provisto por el proveedor que replica el comportamiento de su sistema productivo sin procesar transacciones reales. Permite a los desarrolladores verificar la correcta integración de sus sistemas durante la fase de desarrollo, reduciendo el riesgo de errores en producción. (Ammann & Offutt, 2016)
- Integración de sistemas: La integración de sistemas es el proceso de conectar subsistemas o componentes independientes para que funcionen como un conjunto coordinado. En arquitecturas orientadas a servicios, la integración implica gestionar diferencias en protocolos, formatos de datos y modelos de comunicación entre sistemas que no fueron diseñados para interoperar directamente.(*Hohpe, G. and Woolf, B. (2003) Enterprise Integration Patterns Designing, Building, and Deploying Messaging Solutions. Addison-Wesley. - References - Scientific Research Publishing*, s. f.)

5.2 Análisis de alternativas de solución

5.2.1 Alternativas de solución e impacto

El problema de la fragmentación en los ecosistemas de integración de pagos no es exclusivo del contexto colombiano. En distintas regiones del mundo se han desarrollado aproximaciones para enfrentar la complejidad que surge cuando múltiples proveedores de pago coexisten con convenciones técnicas distintas. A continuación se presentan tres trabajos relevantes que abordan problemáticas similares a las que motivan este proyecto, y cuyo análisis permite comprender el estado del arte en la materia.

Integración de múltiples pasarelas de pago para escenarios de pago dividido (Manuaba & Hisyam, 2022)

En indonesia, el mercado de pagos digitales presenta una fragmentación similar a la del contexto colombiano: distintos proveedores de billeteras electrónicas operan a través de pasarelas separadas, cada una con sus propias APIs, mecanismos de autenticación y formatos de respuesta diferentes. Ante esta situación, Manuaba e Hisyam (2022) desarrollaron un prototipo funcional que integra simultáneamente dos pasarela de pago (*Midtrans* y *Xendit*) en una única aplicación web, con el objetivo de habilitar un escenario donde el pago es dividido entre dos métodos de pago provistos por proveedores distintos.

La solución técnica implementada se basa en el método de integración por *Core API,* que consiste en consumir directamente las APIs REST de cada pasarela sin depender de interfaces visuales pre-diseñadas por los proveedores. Esto otorga control al desarrollador sobre la lógica de autenticación, el formato de las solicitudes y la presentación de la interfaz de pago. El sistema desarrollado utiliza Laravel como capa *backend*, y cada solicitud de pago es enrutada hacia la pasarela correspondiente con el método de pago seleccionado por el usuario. Ambas pasarelas se integran de forma asíncrona cuando se trata de pagos divididos, el sistema almacena el historial de transacciones en una base de datos MySQL para gestionar los callbacks de confirmación en tiempo real.

En términos de impacto técnico, este trabajo demuestra la viabilidad de unificar múltiples proveedores de pago bajo una sola capa de integración, un principio directamente aplicable al problema que enfrenta el ecosistema colombiano. Para los equipos de desarrollo, el valor de esta aproximación radica en la posibilidad de centralizar la lógica de pagos sin estar restringidos a los métodos que ofrece un solo proveedor. Sin embargo, los propios autores reconocen que su implementación corresponde a un prototipo específico, sin pretensiones de convertirse en un framework reutilizable ni en una herramienta generalizable a otros contextos. El costo inicial de construir una capa de integración propia con Core API es mayor que el de utilizar métodos como Snap, que proveen interfaces prediseñadas; no obstante, esa inversión inicial habilita una mayor capacidad de personalización y extensión a largo plazo.

Plataforma universal de pagos: liberándose del vendor lock-in (Penaganti, 2025)

Una de las consecuencias más costosas de operar en ecosistemas de pago fragmentados es el denominado *vendor lock-in*: la dependencia técnica profunda hacia un único proveedor que hace que cualquier migración o cambio implique esfuerzos de desarrollo significativos. Penaganti (2025) aborda este problema mediante la propuesta de una arquitectura de plataforma universal de pagos, orientada a organizaciones empresariales que requieren integrar múltiples proveedores de forma flexible y sin modificar su lógica de negocio central.

La solución propuesta se organiza en tres capas funcionales. La primera es una capa de abstracción que actúa como intermediario entre la aplicación de negocio y los proveedores de pago: recibe solicitudes en un formato estandarizado y las traduce al formato específico de cada pasarela, normalizando también las respuestas de vuelta. La segunda es un motor de enrutamiento dinámico que determina, con base en múltiples criterios como el monto de la transacción, la región geográfica, el nivel de riesgo o la disponibilidad del proveedor, cuál pasarela debe procesar cada pago. La tercera es un sistema de configuración unificada que centraliza las reglas de precios y descuentos, evitando inconsistencias entre proveedores. Adicionalmente, la arquitectura incorpora patrones de resiliencia como circuit breakers y rutas de respaldo para garantizar la disponibilidad del servicio ante fallas parciales.

El impacto de esta arquitectura es sustancial tanto a nivel técnico como económico. Penaganti (2025) documenta que, en implementaciones reales, la migración de un proveedor a otro que normalmente requería entre tres y seis meses de trabajo de desarrollo pudo completarse en aproximadamente dos horas, dado que el cambio opera a nivel de configuración y no de código. A nivel de costos operativos, el enrutamiento inteligente permitió a una plataforma de comercio electrónico reducir sus comisiones de procesamiento en un 23%, al dirigir transacciones pequeñas hacia proveedores con tarifas fijas bajas y transacciones grandes hacia proveedores con mejores tarifas porcentuales. Para los equipos de ingeniería, esta aproximación reduce drásticamente la deuda técnica asociada a las integraciones de pago y permite incorporar nuevos proveedores sin intervenir en la lógica de la aplicación. Desde una perspectiva social y organizacional, la simplificación de la reconciliación contable y la visibilidad centralizada de las transacciones mejoran la eficiencia operativa de las áreas financieras y de atención al cliente.

Interfaz de Pago Unificada (UPI): innovación digital e inclusión financiera (Rastogi et al., 2021)

Mientras que las dos alternativas anteriores abordan el problema desde la perspectiva del desarrollador, esta aproximación enfrenta el contexto desde una perspectiva institucional. La Interfaz de Pago Unificada de India, conocida como UPI, es una plataforma de pagos

digitales desarrollada y regulada por el gobierno indio que opera sobre una arquitectura de Open API. Su objetivo central es ofrecer un canal de pago instantáneo, accesible y de bajo costo que funcione como infraestructura común para todos los participantes del ecosistema financiero, desde grandes bancos hasta pequeñas empresas fintech.

Técnicamente, UPI funciona como una plataforma interoperable en la que distintas aplicaciones y entidades financieras pueden integrarse mediante una interfaz estándar. Esto elimina la necesidad de que cada proveedor construya integraciones bilaterales con todos los demás, reemplazándolas por una capa de comunicación centralizada. La plataforma está diseñada para operar incluso sin conexión a internet permanente, y está vinculada al sistema nacional de identificación, lo que amplía su alcance hacia sectores de la población no bancarizados.

El impacto documentado por Rastogi et al. (2021) trasciende lo técnico. A través de un modelo de ecuaciones estructurales aplicado a datos reales, los autores encontraron que UPI tiene un efecto positivo y estadísticamente significativo sobre la alfabetización financiera de sus usuarios, y que dicha alfabetización, a su vez, contribuye a la inclusión financiera y al desarrollo económico de poblaciones vulnerables. La estabilidad financiera actúa como mediadora entre la alfabetización y la inclusión, mientras que la confianza en el sistema media la relación entre inclusión y desarrollo económico. En términos prácticos, esto significa que una plataforma de pagos unificada no sólo simplifica el trabajo técnico de quienes integran sus sistemas, sino que puede tener efectos amplificados sobre el bienestar económico de las comunidades que la adoptan. Para los desarrolladores y las organizaciones, la existencia de una interfaz estándar respaldada institucionalmente reduce la incertidumbre técnica y normativa, y elimina gran parte del trabajo de integración diferenciada que caracteriza a los ecosistemas fragmentados.

5.2.2 Comparación de alternativas

Las tres alternativas exploradas en la sección anterior comparten con el Kit Pagos Colombia una preocupación central: la complejidad que surge cuando un sistema de software debe interactuar con múltiples proveedores de pago que no hablan el mismo idioma a nivel técnico. Sin embargo, cada una responde a ese problema desde un punto de partida distinto, con alcances, audiencias y contextos de aplicación diferentes. Este apartado examina esas diferencias con el objetivo de situar la propuesta del Kit Pagos Colombia dentro del espacio de soluciones existentes y argumentar por qué su diseño responde de manera más específica al problema identificado anteriormente en la sección 1.1.1.

El trabajo de Manuaba e Hisyam (2022) es el antecedente técnico más directo de esta propuesta. Su valor principal radica en demostrar que es posible integrar múltiples pasarelas bajo una misma capa de aplicación utilizando el método Core API. No obstante, el resultado de ese trabajo es un prototipo acoplado a un caso de uso concreto y no un framework

desacoplado que otros desarrolladores puedan adoptar en sus propios proyectos. La solución no contempla herramientas para probar la implementación y no ofrece mecanismos de extensión hacia otros proveedores sin intervenir directamente el código. Para un equipo de desarrollo colombiano que busca integrar Wompi o PayU, ese trabajo aporta evidencia conceptual valiosa pero no una herramienta aplicable directamente.

La propuesta de Penaganti (2025) representa el extremo opuesto en términos de sofisticación. Su arquitectura de tres capas abstracción, enrutamiento dinámico y configuración unificada es técnicamente robusta y aborda problemas que van más allá de la simple integración, como la optimización de costos por transacción o la tolerancia a fallos mediante circuit breakers. Un circuit breaker es un patrón de resiliencia que interrumpe las solicitudes hacia un servicio cuando este presenta fallas reiteradas, evitando que esos errores se propaguen al resto del sistema. Sin embargo, esta solución está orientada a organizaciones empresariales con infraestructura considerable y equipos de ingeniería especializados. Su implementación asume un nivel de madurez técnica y operativa que no necesariamente corresponde al perfil del desarrollador que enfrenta por primera vez la integración de una pasarela de pago. Adicionalmente, la solución no contempla el contexto normativo ni los proveedores del mercado local colombiano, y no incluye componentes de simulación y prueba orientados al desarrollo temprano.

El caso de UPI (Rastogi et al., 2021) opera en una dimensión diferente a las anteriores. No es una herramienta para desarrolladores sino una infraestructura de pagos de escala nacional, promovida institucionalmente por el gobierno indio. Su impacto sobre la inclusión financiera y el desarrollo económico está bien documentado, pero replicar ese modelo en Colombia implicaría intervenciones de política pública, coordinación entre entidades regulatorias y una adopción generalizada por parte del sistema financiero, elementos que están completamente fuera del alcance de un proyecto de software académico. Desde la perspectiva del desarrollador individual o del equipo pequeño, UPI no es una alternativa comparable: es un ecosistema que alguien más construyó y sobre el cual otros construyen.

El Kit Pagos Colombia se diferencia de estas tres alternativas por su enfoque en el desarrollador colombiano como usuario principal, y por combinar en un solo framework componentes que las soluciones anteriores no reúnen simultáneamente: una SDK con interfaz unificada para las pasarelas dominantes en el mercado local, una API de simulación desplegada que no depende de credenciales externas, y documentación centralizada de datos de prueba válidos para el contexto colombiano. Ninguna de las alternativas analizadas ofrece los tres componentes de forma integrada, lo que representa la contribución diferencial de esta propuesta.

Es importante reconocer, sin embargo, que el Kit Pagos Colombia tiene limitaciones que las alternativas anteriores no tienen. Penaganti (2025) contempla mecanismos de resiliencia avanzados y optimización de enrutamiento que están fuera del alcance de esta propuesta. UPI,

por su naturaleza institucional, garantiza una interoperabilidad sistémica que ningún framework de desarrollo puede replicar por sí solo. Y aunque el trabajo de Manuaba e Hisyam (2022) es más acotado, fue validado en un entorno de producción sandbox real con resultados medibles, algo que el Kit Pagos Colombia deberá demostrar en su fase de evaluación.

Criterio	Manuaba & Hisyam (2022)	Penaganti (2025)	Rastogi et al. (2021)	Kit Pagos Colombia
Contexto geográfico	Indonesia	Global / empresarial	India	Colombia
Orientación principal	Prototipo de aplicación	Arquitectura empresarial	Política pública	Herramienta para desarrolladores
Reutilizable como framework	No	Parcial (requiere infraestructura)	No aplica	Sí
API de simulación local	No	No	No aplica	Sí
Datos de prueba consolidados	No	No	No aplica	Sí
Cobertura de pasarelas colombianas	No	No	No aplica	Sí (Wompi, PayU, Mercadopago, Kushki)
Open source	No especificado	No especificado	No aplica	Sí
Resiliencia y enrutamiento dinámico	No	Sí	No aplica	No

Validación empírica documentada	Sí (sandbox)	Sí (casos reales)	Sí (SEM)	Pendiente (fase de evaluación)
Complejidad de adopción para equipos pequeños	Media	Alta	No aplica	Baja
Respaldo institucional	No	No	Sí	No

6 Referencias

Ammann, P., & Offutt, J. (2016). Introduction to software testing. Cambridge University Press. https://doi.org/10.1017/9781316771273

Amazon Web Services. (s. f.). ¿Qué es un marco en programación e ingeniería? https://aws.amazon.com/es/what-is/framework/

Cockburn, A. (2024). Hexagonal architecture explained.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Tesis doctoral, University of California, Irvine).

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Hisyam, M., & Manuaba, I. B. K. (2022). Integration model of multiple payment gateways for online split payment scenario. En 2022 International Conference on Information Management and Technology (ICIMTech) (pp. 122–126). https://doi.org/10.1109/ICIMTech55957.2022.9915168

Hohpe, G., & Woolf, B. (2003). Enterprise integration patterns: Designing, building, and deploying messaging solutions. Addison-Wesley.

Informe de comercio minorista / diciembre 2025. (s. f.). Cámara Colombiana de Comercio Electrónico. Recuperado el 16 de marzo de 2026, de https://ccce.org.co/noticias/informe-de-comercio-minorista/

IEEE. (1998). IEEE recommended practice for software requirements specifications (IEEE Std 830-1998). https://doi.org/10.1109/IEEESTD.1998.88286

Peffers, K., Tuunanen, T., Rothenberger, M., & Chatterjee, S. (2007). A design science research methodology for information systems research. Journal of Management Information Systems, 24(3), 45–77. https://www.researchgate.net/publication/284503626_A_design_science_research_methodology_for_information_systems_research

Taherkordi, A., Romero, D., Rouvoy, R., & Eliassen, F. (2011). REST: From research to practice.

Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object-oriented design. IEEE Transactions on Software Engineering, 20(6), 476–493. https://doi.org/10.1109/32.295895

Hexagonal architecture pattern—AWS Prescriptive Guidance. (s. f.). Amazon Web Services. Recuperado el 15 de abril de 2026, de https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html

Patrones de diseño GoF por página. (s. f.). Scribd. Recuperado el 15 de abril de 2026, de https://es.scribd.com/document/388374728/Patrones-de-Diseno-GoF-Por-Pagina